FinSight

AI-Powered Financial Intelligence Platform

Architecture & Technical Reference | FYP Documentation

Gemini 2.5 Flash | Next.js 14.2 | Flask | ReportLab | PyMuPDF | Pydantic

01 The Problem Being Solved

Analyzing corporate annual reports from the **Pakistan Stock Exchange (PSX)** is intensely manual, slow, and error-prone. These reports are typically **200+ pages long**, embedded with marketing content, and feature complex tabular data often in localized numeric formatting.

Financial analysts waste hours hunting for the Balance Sheet, standardizing figures, and copying data to Excel before any analysis can begin. **FinSight eliminates this entirely.**

Pain Point	Impact	FinSight Solution
Manual page hunting	Hours wasted per report	Keyword density radar auto-isolates pages
Localized number formats (e.g. 14 500 000)	Parsing errors	Regex sanitization → clean integers
Scanned / image PDFs	No digital text to extract	Gemini Vision Pipeline reads like a human
Ratio calculation errors	Incorrect analysis	Algebraic math engine with strict formulas
Manual narrative writing	Bottleneck for analysts	AI generates 3-section summary in <3 s

02 Processing Overview: PDF to Analytics

From the moment a user uploads a PDF, FinSight runs a **7-stage automated pipeline** that transforms raw, unstructured document bytes into a structured analytical dashboard and a downloadable institutional report.

#	Stage	What Happens
1	Ingestion	User drops PDF into Next.js frontend; sent via POST to Flask
2	Filtration	Keyword radar scans every page; discards non-financial content
3	Extraction & Sanitization	Text or vision pipeline reads data; regex cleans number formats

#	Stage	What Happens
4	AI Structuring	Gemini maps raw data → strict JSON (Assets, Liabilities, Revenue…)
5	Computation	Math engine calculates liquidity, profitability, leverage & efficiency
6	Analysis & Flagging	Thresholds checked; dangerous ratios trigger CRITICAL badges
7	Delivery	Dashboard rendered live; 12-page PDF dossier compiled & downloadable

03 System Outputs

FinSight produces two complementary outputs simultaneously on processing completion:

Live Dashboard

- Colour-coded metric scorecards
- Comparative data tables
- Red Flags triage centre
- AI executive narrative

12-Page PDF Dossier

- Extracted Balance Sheet
- Income & Cash Flow statements
- Diagnostic exception arrays
- Ratio analysis tables

04 Core Processing Operations

Keyword Density Radar

Counts financial keywords on every page to mathematically score and rank pages. Pages scoring below threshold are discarded; only the core 5–15 statements survive.

Structural Number Sanitization

Regex engine strips PSX formatting artifacts. Input: `15 423 000` → Output: `15423000`. Ensures the AI receives clean, unambiguous numeric strings.

Algebraic Ratio Computation

Strict formula engine calculates all key ratios. Example: Current Ratio = Current Assets ÷ Current Liabilities. No floating-point approximations; division-by-zero guards included.

Automated Threshold Diagnostics

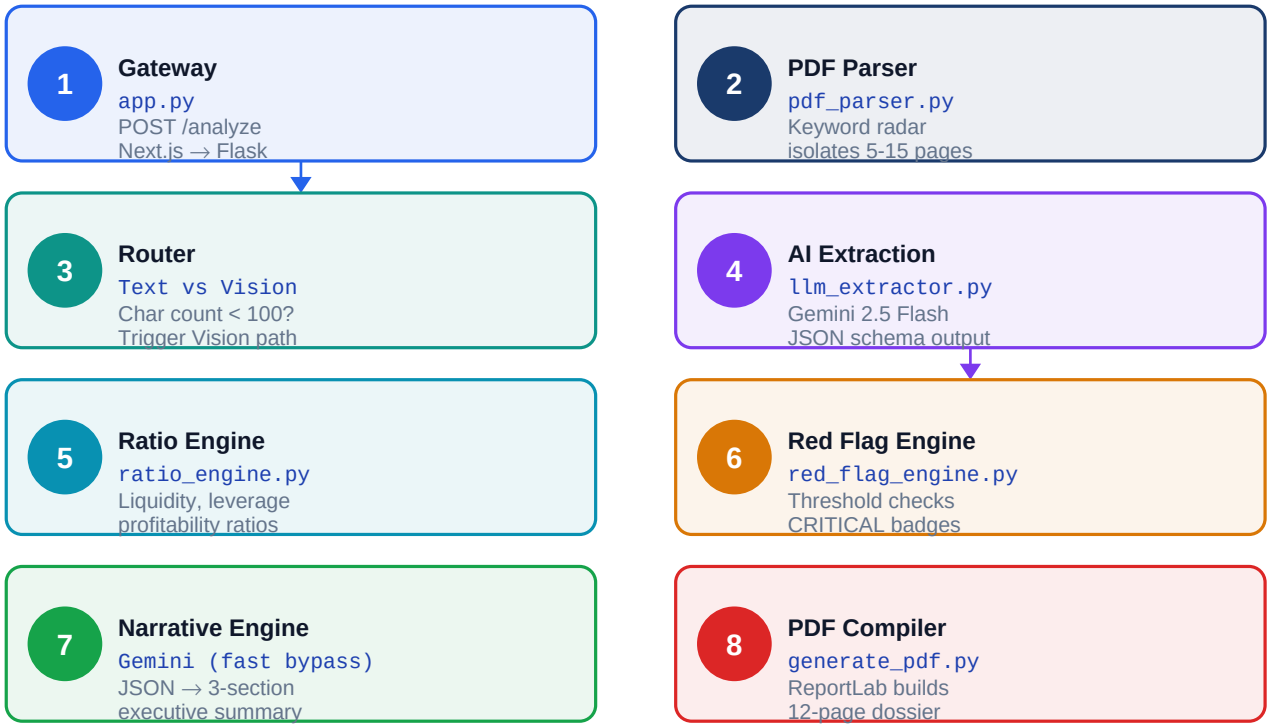
Computed ratios compared against pre-set safety bounds. Interest Coverage < 1.5× → programmatic **[CRITICAL]** risk badge. Severity tiers: CRITICAL, HIGH, MEDIUM, LOW.

High-Speed Narrative Generation

Instead of re-feeding the 200-page PDF, the extracted lightweight JSON is sent to Gemini. Result: a 3-section executive summary generated in under 3 seconds.

05 Backend Flow & Architecture

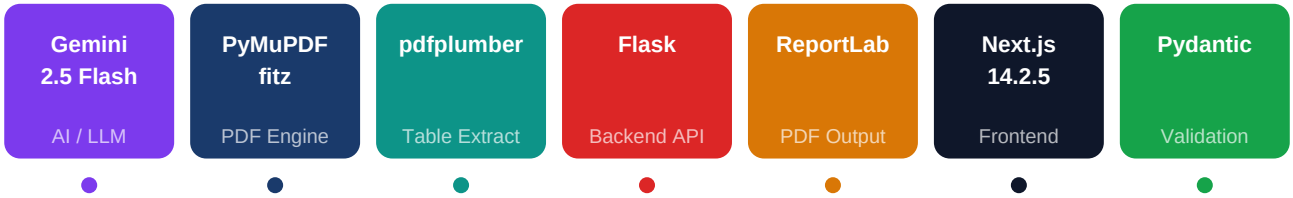
Below is the exact journey of a file through all architectural layers, from the initial HTTP request to the final JSON response delivered to the frontend.



File & Module Map

File / Module	Layer	Responsibility
app.py	Gateway	Receives POST /analyze, orchestrates the full pipeline, handles multi-attempt fa
core/pdf_parser.py	Parsing	Keyword radar, page isolation, dynamic Text vs Vision routing decision (200 DP
core/llm_extractor.py	AI Extraction	Calls Gemini with exponential backoff on rate limits; enforces Pydantic JSON sc
core/ratio_engine.py	Math	Calculates all financial ratios algebraically from clean JSON numbers.
core/red_flag_engine.py	Diagnostics	Evaluates ratios against boundaries; assigns standardized severity exception ba
generate_pdf.py	Output	ReportLab dynamically draws and compiles the 12-page downloadable dossier.

06 Libraries, Frameworks & AI Model



Library / Tool	Version / Model	Purpose
Google Gemini	2.5 Flash	High-speed AI extraction (text + vision pipelines) & narrative generation. 1M token c
PyMuPDF (fitz)	Latest	Page isolation, text layer reading, and RAM-based 200 DPI PNG rasterization for Vi
pdfplumber	Latest	Fallback structured table boundaries mapping for complex unreadable PSX tabular
Flask	Latest	Python WSGI backend processing engine managing lifecycle boundaries and expor
ReportLab	Latest	Programmatic PDF generation module, rendering path-based custom flowables for
Next.js	14.2.5	Frontend SSR dashboard scaling. Configured with extended Node memory allocatio
Pydantic	v2	Type-validation layer ensuring strict JSON schema mappings so the math engine ne

07 Automatic Multimodal Fallback Pipeline

One of FinSight's strongest technical differentiators: it handles **both readable and scanned PDFs** without any user intervention. The routing decision is calculated per-document dynamically.

Characteristic	Text Pipeline	Vision Pipeline
Trigger condition	char count >= 100 per page	char count < 100 per page
OCR library used	None (pure text scrape)	None — Gemini Vision natively interprets pixels
Image conversion	Not needed	Targeted 200 DPI PNG, in-memory (RAM only)
Processing Speed	Fastest (~2-4 seconds)	Slightly slower (Vision encoding + payload chunking)
Accuracy on tables	High	Near-perfect (Gemini perceives physical layout structurally)

Characteristic	Text Pipeline	Vision Pipeline
Dependency required	Local Python environment	Google API connection